Segmentación automática de regiones en imágenes hiperespectrales

Código completo de Python y explicación matemática paso a paso

21 de octubre de 2026

Índice

Parte I. Código completo: `Segmentacion_Mural_SLIC_SAM.py` 3

Parte II. Explicación completa para estudiar y presentar 11

Explicación del algoritmo de segmentación automática SLIC + SAM para imágenes hiperespectrales de murales 11

1. ¿Qué problema estamos intentando resolver? 11

2. Primera etapa: cargar el cubo hiperespectral 11
¿Qué recibe el programa? . . . 12

3. Segunda etapa: construir una imagen de falso RGB 12
¿Por qué no utilizamos las 128 bandas directamente? . . . 13

4. Tercera etapa: segmentación mediante SLIC 13
4.1. ¿Qué es SLIC? . . . 13
4.2. ¿Cómo funciona matemáticamente? . . . 14
4.3. ¿Qué significa el parámetro de compactación? . . . 14
4.4. ¿Cuántos superpíxeles genera? . . . 15

5. Cuarta etapa: calcular un espectro representativo para cada superpíxel 15
5.1. ¿Cómo obtenemos el espectro de una región? . . . 15

6. Quinta etapa: normalización de los vectores espectrales 16
¿Por qué hacemos esto? . . . 16

7. Sexta etapa: comparación espectral mediante SAM 17
7.1. ¿Qué es SAM? . . . 17
7.2. Interpretación física . . . 17

8. Séptima etapa: encontrar qué regiones son vecinas 18
¿Por qué hacemos esto? . . . 19

9. Octava etapa: fusión de regiones mediante SAM 19
9.1. Primer criterio: similitud espectral . . . 19
9.2. Segundo criterio: dispersión espectral acumulada . . . 20

10. Novena etapa: ¿cómo decide el orden de las fusiones? 21
¿Cuándo termina? . . . 21

11. Décima etapa: eliminar píxeles de los bordes 22
¿Por qué? . . . 22

¿Cómo elimina los bordes? . . . 22

12. Undécima etapa: filtrado espectral de cada píxel 23
12.1. Calcular la referencia espectral . . . 23
12.2. Comparar cada píxel . . . 23

13. Duodécima etapa: eliminar regiones pequeñas 24

14. Decimotercera etapa: asignar etiquetas automáticamente 24
¿Qué representa entonces una región? . . . 25

15. Decimocuarta etapa: seleccionar espectros para exportar 25
¿Por qué limitar el número de espectros? . . . 25

16. Decimoquinta etapa: guardar los resultados 26
A. Archivo Parquet . . . 26
B. Archivo JPG . . . 26
C. Archivo CSV . . . 26
D. Archivo JSON . . . 27

17. ¿Cómo interpretar el resultado de tu mural? 27

18. ¿Cómo se relaciona con tus dos flujos de trabajo? 28
Flujo 1 . . . 28
Flujo 2 . . . 28
¿Por qué es metodológicamente interesante? . . . 28

19. ¿Qué limitaciones tiene el algoritmo actual? 29

20. ¿Qué podrías presentar como avances? 29
1. Problema . . . 29
2. Propuesta . . . 29
3. Implementación . . . 29
4. Resultados preliminares . . . 30
5. Trabajo pendiente . . . 30

21. Una explicación breve que puedes darle a tu asesor 30

22. Tres preguntas que probablemente te haga tu asesor 30
Pregunta 1. ¿Por qué utilizaste SLIC y SAM juntos? . . . 31
Pregunta 2. ¿Cómo sabes que realmente identificaste pigmentos? . . . 31
Pregunta 3. ¿Cómo sabes que 35 regiones es el número correcto? . . . 31

23. Lo que considero más importante para tu tesis 31
Recomendación para la reunión . . . 31

Parte I. Código completo: `Segmentacion_Mural_SLIC_SAM.py`

A continuación aparece el **archivo Python completo (566 líneas)**, sin omisiones ni modificaciones. Es la versión compartida en esta conversación que produjo la segmentación de regiones del mural.

```
"""Segmentacion NO supervisada de una region de mural hiperespectral.

ENTRADA: UN cubo ENVI (.hdr asociado a .img/.dat/.raw) YA preprocesado
         con normalizacion + MNF + MNF inverso. Tambien se admite --hdr.
         Si solo quieres un fragmento del mural, entrega el cubo ENVI
         correspondiente a ese fragmento; NO se solicita seleccionar ROI a mano.

PROCESO:
    1. SLIC crea superpixeles a partir de una visualizacion de falso RGB.
    2. Se calcula un espectro medio por superpixel usando el cubo completo.
    3. Se fusionan SOLO superpixeles vecinos cuyos espectros cumplen un
       umbral SAM y un limite de dispersion angular acumulada.
    4. Se excluyen regiones pequenas, pixeles de frontera y pixeles con
       SAM alto respecto al espectro representativo de su region.
    5. Se etiquetan las regiones resultantes como Region_0001, ...

El NUMERO DE REGIONES NO SE FIJA A PRIORI. Depende de los umbrales, la
resolucion de SLIC, la escena y la calidad del cubo. Las etiquetas NO son
identificaciones quimicas ni clases de pigmentos confirmadas.

SALIDAS: Parquet de espectros, JPG de las mascaras, CSV de regiones y JSON
         de parametros. Columnas compatibles con el selector de la tabla:
         clase, pixel_id, coord_x, coord_y, espectro, imagen_id, region_id.

Dependencias (una sola vez):
    pip install numpy scipy scikit-image matplotlib spectral pyarrow

EJEMPLOS:
    python Segmentacion_Mural_SLIC_SAM.py
    python Segmentacion_Mural_SLIC_SAM.py --hdr "mural_MNF_inverso.hdr" --salida "resultados"
    python Segmentacion_Mural_SLIC_SAM.py --segmentos 1800 --angulo-fusion 4 --angulo-pixel 8
    python Segmentacion_Mural_SLIC_SAM.py --max-pixeles-region 0  # todos los espectros

Importante: evitar usar el resultado como etiquetas verdaderas para medir
la exactitud de una CNN; se requiere validacion independiente.
"""

from __future__ import annotations

import argparse
import csv
import heapq
import json
from pathlib import Path

import matplotlib
import numpy as np
from scipy.ndimage import maximum_filter, minimum_filter, find_objects
from skimage.segmentation import find_boundaries, slic


def archivos_entrada_salida(hdr: str | None, salida: str | None) -> tuple[Path, Path]:
    """Unicamente una imagen ENVI de entrada y la carpeta DESTINO de resultados."""
    if not hdr or not salida:
        import tkinter as tk
        from tkinter import filedialog
        raiz = tk.Tk()
        raiz.withdraw()
        raiz.attributes('-topmost', True)
        try:
            if not hdr:
                hdr = filedialog.askopenfilename(
                    parent=raiz,
                    title='ENTRADA: UN cubo de mural despues de MNF inverso (.hdr)',
                    filetypes=[('Cabeceras ENVI', '*.hdr'), ('Todos', '*.*')],
                )
            if not hdr:
                raise SystemExit('No seleccionaste la imagen hiperespectral.')
            if not salida:
```

```
                salida = filedialog.askdirectory(
                    parent=raiz,
                    title='SALIDA: carpeta para guardar Parquet, JPG, CSV y JSON',
                )
            if not salida:
                raise SystemExit('No seleccionaste carpeta de salida.')
        finally:
            raiz.destroy()

    entrada = Path(hdr)
    if entrada.suffix.lower() != '.hdr':
        entrada = entrada.with_suffix('.hdr')
    if not entrada.is_file():
        raise FileNotFoundError(f'No existe la cabecera ENVI: {entrada}')
    return entrada, Path(salida)


def cargar_cubo(ruta_hdr: Path) -> np.ndarray:
    from spectral import envi
    imagen = envi.open(str(ruta_hdr))
    cubo = np.asarray(imagen.load(), dtype=np.float32)
    if cubo.ndim != 3:
        raise ValueError('Se esperaba un cubo de [filas, columnas, bandas].')
    print(f'Entrada MNF inverso: {cubo.shape}', flush=True)
    return cubo


def falso_rgb(cubo: np.ndarray, bandas: tuple[int, int, int]) -> np.ndarray:
    """Transformacion SOLO para visualizacion/SLIC, no altera espectros SAM."""
    alto, ancho, n = cubo.shape
    if any(b < 0 or b >= n for b in bandas):
        raise ValueError(f'Bandas {bandas} invalidas; la imagen tiene {n} bandas.')
    rgb = np.empty((alto, ancho, 3), dtype=np.float32)
    for j, banda in enumerate(bandas):
        canal = cubo[:, :, banda]
        validos = canal[np.isfinite(canal)]
        if not validos.size:
            rgb[:, :, j] = 0.0
            continue
        p2, p98 = np.percentile(validos, [2, 98])
        limpio = np.nan_to_num(canal, nan=float(p2), posinf=float(p98), neginf=float(p2))
        rgb[:, :, j] = np.clip((limpio - p2) / max(float(p98 - p2), 1e-8), 0, 1)
    return rgb


def espectros_superpixeles(cubo: np.ndarray, etiquetas: np.ndarray,
                           max_muestra: int = 300_000) -> tuple[np.ndarray, np.ndarray]:
    """Promedios por superpixel a partir de una reticula muestreada.

    Se evita construir una copia grande de toda la imagen o una lista por pixel.
    El numero de muestras define la aproximacion de los espectros iniciales.
    """
    h, w, n = cubo.shape
    n_seg = int(etiquetas.max()) + 1
    salto = max(1, int(np.ceil(np.sqrt(h * w / max_muestra))))
    pixels = cubo[::salto, ::salto, :].reshape(-1, n)
    ids = etiquetas[::salto, ::salto].ravel()
    validos = np.isfinite(pixels).all(axis=1)
    # Se descartan espectros nulos (o de magnitud casi nula).
    validos &= np.linalg.norm(np.nan_to_num(pixels, copy=True), axis=1) > 1e-10
    ids_v = ids[validos]
    pix_v = pixels[validos]
    cuentas = np.bincount(ids_v, minlength=n_seg).astype(np.float64)
    promedios = np.zeros((n_seg, n), dtype=np.float64)
    for j in range(n):
        promedios[:, j] = np.bincount(ids_v, weights=pix_v[:, j], minlength=n_seg)
    np.divide(promedios, np.maximum(cuentas[:, None], 1), out=promedios)
    norms = np.linalg.norm(promedios, axis=1)
    buenos = (cuentas > 0) & (norms > 1e-10)
    promedios[buenos] /= norms[buenos, None]
    promedios[~buenos] = 0
    return promedios.astype(np.float32), cuentas


def pares_adyacentes(etiquetas: np.ndarray) -> np.ndarray:
    """Pares no dirigidos de etiquetas que comparten frontera 4-conexa."""
    base = int(etiquetas.max()) + 1
```

```
    result = []
    for a, b in ((etiquetas[:, :-1], etiquetas[:, 1:]),
                 (etiquetas[:-1, :], etiquetas[1:, :])):
        distinto = a != b
        x = a[distinto].astype(np.int64)
        y = b[distinto].astype(np.int64)
        if x.size:
            result.append(np.minimum(x, y) * base + np.maximum(x, y))
    if not result:
        return np.empty((0, 2), dtype=np.int32)
    codigos = np.unique(np.concatenate(result))
    return np.column_stack((codigos // base, codigos % base)).astype(np.int32)


def fusion_sam(etiquetas: np.ndarray, referencias: np.ndarray, cuentas: np.ndarray,
               angulo_fusion: float, dispersion_max: float) -> np.ndarray:
    """Fusion aglomerativa de regiones ADYACENTES.

    Recalcula el espectro representativo despues de cada fusion. El limite de
    dispersion (cota conservadora, via desigualdad triangular angular) evita
    uniones en cadena entre zonas que terminan siendo demasiado distintas.
    """
    n = len(referencias)
    padre = np.arange(n, dtype=np.int32)
    peso = np.maximum(cuentas.astype(np.float64), 1)
    acumulado = referencias.astype(np.float64) * peso[:, None]
    norma = np.linalg.norm(acumulado, axis=1)
    unit = np.zeros_like(acumulado)
    activo = (cuentas > 0) & (norma > 1e-10)
    unit[activo] = acumulado[activo] / norma[activo, None]
    radio = np.zeros(n, dtype=np.float64)
    vecinos: list[set[int]] = [set() for _ in range(n)]
    pares = pares_adyacentes(etiquetas)
    for a, b in pares:
        a, b = int(a), int(b)
        vecinos[a].add(b)
        vecinos[b].add(a)

    def raiz(x: int) -> int:
        while padre[x] != x:
            padre[x] = padre[padre[x]]
            x = int(padre[x])
        return x

    def angulo(a: int, b: int) -> float:
        return float(np.degrees(np.arccos(np.clip(np.dot(unit[a], unit[b]), -1, 1))))

    cola: list[tuple[float, int, int]] = []
    for a, b in pares:
        a, b = int(a), int(b)
        if activo[a] and activo[b]:
            heapq.heappush(cola, (angulo(a, b), a, b))

    fusionados = 0
    while cola:
        viejo_angulo, ia, ib = heapq.heappop(cola)
        a, b = raiz(ia), raiz(ib)
        if a == b or not (activo[a] and activo[b]):
            continue
        if b not in vecinos[a]:
            continue
        actual = angulo(a, b)
        # Una arista puede cambiar cuando se fusiona alguno de sus extremos.
        if abs(actual - viejo_angulo) > 1e-5:
            heapq.heappush(cola, (actual, a, b))
            continue
        if actual > angulo_fusion:
            continue

        nuevo_sum = acumulado[a] + acumulado[b]
        nuevo_norm = np.linalg.norm(nuevo_sum)
        if nuevo_norm < 1e-10:
            continue
        nuevo_u = nuevo_sum / nuevo_norm
        desplaz_a = float(np.degrees(np.arccos(np.clip(np.dot(unit[a], nuevo_u), -1, 1))))
        desplaz_b = float(np.degrees(np.arccos(np.clip(np.dot(unit[b], nuevo_u), -1, 1))))
        nuevo_radio = max(radio[a] + desplaz_a, radio[b] + desplaz_b)
```

```
        if nuevo_radio > dispersion_max:
            continue

        if peso[a] < peso[b]:
            a, b = b, a
        padre[b] = a
        peso[a] += peso[b]
        acumulado[a] = nuevo_sum
        unit[a] = nuevo_u
        radio[a] = nuevo_radio
        activo[b] = False

        antiguos = vecinos[a] | vecinos[b]
        antiguos.discard(a)
        antiguos.discard(b)
        vecinos[a] = set()
        vecinos[b].clear()
        for v in antiguos:
            v = raiz(v)
            if v == a:
                continue
            vecinos[a].add(v)
            vecinos[v].discard(b)
            vecinos[v].add(a)
            if activo[v]:
                heapq.heappush(cola, (angulo(a, v), min(a, v), max(a, v)))
        fusionados += 1
    print(f'  Uniones espectrales: {fusionados}', flush=True)
    raices = np.array([raiz(i) for i in range(n)], dtype=np.int32)
    return raices


def mapa_inicial(cubo: np.ndarray, rgb: np.ndarray, n_segmentos: int,
                 compactacion: float, sigma: float, angulo_fusion: float,
                 dispersion_max: float) -> tuple[np.ndarray, int]:
    print(f'SLIC: solicitando ~{n_segmentos} superpixeles...', flush=True)
    # slic trabaja sobre la visualizacion, no sobre el cubo de 128 bandas.
    etiquetas = slic(rgb, n_segments=n_segmentos,
                     compactness=compactacion, sigma=sigma,
                     start_label=0, channel_axis=-1,
                     convert2lab=True, enforce_connectivity=True).astype(np.int32)
    num = int(np.unique(etiquetas).size)
    print(f'  Superpixeles producidos: {num}', flush=True)
    refs, cuentas = espectros_superpixeles(cubo, etiquetas)
    print('Fusionando vecinos segun SAM (sin fijar numero de regiones)...', flush=True)
    raices = fusion_sam(etiquetas, refs, cuentas, angulo_fusion, dispersion_max)
    _, compactos = np.unique(raices, return_inverse=True)
    mapa = compactos[etiquetas] + 1
    return mapa.astype(np.int32), num


def mascaras_espectrales(cubo: np.ndarray, mapa: np.ndarray,
                         borde: int, angulo_pixel: float, min_area: int,
                         max_muestra: int = 300_000) -> tuple[np.ndarray, list[dict]]:
    """Retiene pixel interior con SAM compatible a su region.

    Referencias estimadas de manera aproximada con muestreo de los pixeles
    interiores. SAM se calcula en lotes: no se duplica el cubo completo.
    """
    h, w, bandas = cubo.shape
    n_reg = int(mapa.max())
    tamanos = np.bincount(mapa.ravel(), minlength=n_reg + 1)
    print(f'  Regiones antes de filtrar por tamano: {n_reg}', flush=True)

    if borde > 0:
        tam = 2 * borde + 1
        interior = (minimum_filter(mapa, size=tam, mode='nearest') ==
                    maximum_filter(mapa, size=tam, mode='nearest'))
    else:
        interior = np.ones((h, w), dtype=bool)
    candidatos = interior & (tamanos[mapa] >= min_area)

    # Referencia media robustificada frente a periferias: solo pixeles interiores.
    paso = max(1, int(np.ceil(np.sqrt(h * w / max_muestra))))
    sub_c = cubo[::paso, ::paso, :].reshape(-1, bandas)
    sub_ids = mapa[::paso, ::paso].ravel()
    sub_masc = candidatos[::paso, ::paso].ravel()
```

```
    buenos = sub_masc & np.isfinite(sub_c).all(axis=1)
    buenos &= np.linalg.norm(np.nan_to_num(sub_c, copy=True), axis=1) > 1e-10
    ii, xx = sub_ids[buenos], sub_c[buenos]
    cuenta = np.bincount(ii, minlength=n_reg + 1)
    refs = np.zeros((n_reg + 1, bandas), dtype=np.float64)
    for j in range(bandas):
        refs[:, j] = np.bincount(ii, weights=xx[:, j], minlength=n_reg + 1)
    np.divide(refs, np.maximum(cuenta[:, None], 1), out=refs)
    norma = np.linalg.norm(refs, axis=1)
    validas = (cuenta > 0) & (norma > 1e-10)
    refs[validas] /= norma[validas, None]
    refs[~validas] = 0.0
    refs = refs.astype(np.float32)

    # Esta operacion es sobre los pixels que cumplen geometria y tamano.
    indices = np.flatnonzero(candidatos.ravel() & validas[mapa.ravel()])
    cubo_plano = cubo.reshape(-1, bandas)
    ids_plano = mapa.ravel()
    seleccion = np.zeros(h * w, dtype=bool)
    lote = 25_000
    for start in range(0, len(indices), lote):
        ind = indices[start:start + lote]
        datos = cubo_plano[ind]
        finitos = np.isfinite(datos).all(axis=1)
        datos = np.nan_to_num(datos, copy=True, nan=0, posinf=0, neginf=0)
        nr = np.linalg.norm(datos, axis=1)
        cos = np.einsum('ij,ij->i', datos, refs[ids_plano[ind]]) / np.maximum(nr, 1e-10)
        cos = np.clip(cos, -1, 1)
        ang = np.degrees(np.arccos(cos))
        ok = finitos & (nr > 1e-10) & (ang <= angulo_pixel)
        seleccion[ind[ok]] = True

    mapa_ok = np.where(seleccion.reshape(h, w), mapa, 0).astype(np.int32)
    seleccionados = np.bincount(mapa_ok.ravel(), minlength=n_reg + 1)
    conservar = np.flatnonzero(seleccionados >= min_area)
    conservar = conservar[conservar != 0]

    # Numeros consecutivos en orden de tamano descendente (ID no quimico).
    conservar = conservar[np.argsort(-seleccionados[conservar], kind='stable')]
    tabla = np.zeros(n_reg + 1, dtype=np.int32)
    tabla[conservar] = np.arange(1, len(conservar) + 1, dtype=np.int32)
    mapa_ok = tabla[mapa_ok]
    # Calcular centros y cajas en UNA pasada; eficiente con muchas regiones.
    yy, xx = np.nonzero(mapa_ok)
    ids_finales = mapa_ok[yy, xx]
    sum_x = np.bincount(ids_finales, weights=xx, minlength=len(conservar) + 1)
    sum_y = np.bincount(ids_finales, weights=yy, minlength=len(conservar) + 1)
    cuenta_final = np.bincount(ids_finales, minlength=len(conservar) + 1)
    cajas = find_objects(mapa_ok)
    resumen = []
    for nuevo_id, viejo_id in enumerate(conservar, 1):
        rect = cajas[nuevo_id - 1]
        resumen.append({
            'region_id': int(nuevo_id),
            'clase': f'Region_{nuevo_id:04d}',
            'pixeles_region_slic_sam': int(tamanos[viejo_id]),
            'pixeles_compatibles': int(cuenta_final[nuevo_id]),
            'pixeles_exportados': 0,
            'centro_x': round(float(sum_x[nuevo_id] / cuenta_final[nuevo_id]), 1),
            'centro_y': round(float(sum_y[nuevo_id] / cuenta_final[nuevo_id]), 1),
            'x_min': int(rect[1].start), 'y_min': int(rect[0].start),
            'x_max': int(rect[1].stop - 1), 'y_max': int(rect[0].stop - 1),
        })
    print(f'  Regiones finales >= {min_area} pixeles: {len(resumen)}', flush=True)
    return mapa_ok, resumen


def elegir_pixeles(mapa: np.ndarray, resumen: list[dict], max_por_region: int,
                   semilla: int) -> np.ndarray:
    """El JPG muestra regiones completas; el Parquet puede muestrear por region."""
    validos = np.flatnonzero(mapa.ravel() > 0)
    orden = np.argsort(mapa.ravel()[validos], kind='stable')
    validos = validos[orden]
    cantidades = np.bincount(mapa.ravel(), minlength=len(resumen) + 1)
    limites = np.cumsum(cantidades[1:])
    rng = np.random.default_rng(semilla)
    elegidos = []
```

```
    inicio = 0
    for item, fin in zip(resumen, limites):
        grupo = validos[inicio:fin]
        inicio = int(fin)
        if max_por_region > 0 and len(grupo) > max_por_region:
            grupo = np.sort(rng.choice(grupo, size=max_por_region, replace=False))
        item['pixeles_exportados'] = int(len(grupo))
        elegidos.append(grupo)
    if not elegidos:
        return np.empty(0, dtype=np.int64)
    return np.concatenate(elegidos)


def figura_resultado(rgb: np.ndarray, mapa: np.ndarray,
                     resumen: list[dict], max_etiquetas: int):
    import matplotlib.pyplot as plt
    overlay = rgb.copy()
    # Tabla de colores vectorizada: evita iterar sobre toda la imagen por region.
    colores = plt.get_cmap('gist_ncar')
    paleta = np.array([colores((rid * 0.61803398875) % 1.0)[:3]
                       for rid in range(len(resumen) + 1)], dtype=np.float32)
    mascara = mapa > 0
    overlay[mascara] = 0.35 * rgb[mascara] + 0.65 * paleta[mapa[mascara]]
    limites = find_boundaries(mapa, mode='inner') & (mapa > 0)
    overlay[limites] = 1.0
    fig, ax = plt.subplots(1, 2, figsize=(16, 8), constrained_layout=True)
    ax[0].imshow(rgb)
    ax[0].set_title('Mural preprocesado (falso RGB)')
    ax[1].imshow(overlay)
    ax[1].set_title(f'Regiones homogeneas detectadas: {len(resumen)} (SLIC + SAM)')
    for r in resumen[:max_etiquetas]:
        ax[1].text(r['centro_x'], r['centro_y'], str(r['region_id']),
                   color='white', ha='center', va='center', fontsize=7,
                   bbox=dict(fc='black', ec='none', alpha=0.70, pad=1.0))
    for a in ax:
        a.axis('off')
    return fig


def guardar(ruta: Path, salida: Path, cubo: np.ndarray,
            mapa: np.ndarray, resumen: list[dict], indices: np.ndarray,
            figura, config: dict) -> list[Path]:
    """Parquet en lotes para no desbordar memoria en cubos grandes."""
    import pyarrow as pa
    import pyarrow.parquet as pq

    salida.mkdir(parents=True, exist_ok=True)
    stem = ruta.stem
    parquet = salida / f'base_datos_{stem}_mural.parquet'
    jpg = salida / f'{stem}_mural_regiones.jpg'
    csv_r = salida / f'{stem}_mural_regiones.csv'
    json_p = salida / f'{stem}_mural_parametros.json'
    rutas = [parquet, jpg, csv_r, json_p]
    if any(p.exists() for p in rutas):
        raise FileExistsError('Ya existe algun archivo de salida. Usa otra carpeta: ' +
                              ', '.join(str(p) for p in rutas if p.exists()))
    if not len(indices):
        raise RuntimeError('No hay pixeles seleccionados; no se creo Parquet.')

    esquema = pa.schema([
        ('clase', pa.string()), ('pixel_id', pa.int64()),
        ('coord_x', pa.int32()), ('coord_y', pa.int32()),
        ('espectro', pa.list_(pa.float32())),
        ('imagen_id', pa.string()), ('region_id', pa.int32()),
    ])
    w = cubo.shape[1]
    bands = cubo.shape[2]
    plano = cubo.reshape(-1, bands)
    plano_mapa = mapa.ravel()
    escritor = None
    try:
        escritor = pq.ParquetWriter(parquet, schema=esquema, compression='zstd')
        paso = 8_000
        for comienzo in range(0, len(indices), paso):
            inds = indices[comienzo:comienzo + paso]
            ids = plano_mapa[inds].astype(np.int32)
            espectros = np.ascontiguousarray(plano[inds], dtype=np.float32)
```

```
            offsets = pa.array(np.arange(0, (len(inds) + 1) * bands, bands, dtype=np.int32))
            lista = pa.ListArray.from_arrays(offsets, pa.array(espectros.ravel(), type=pa.float32()))
            tabla = pa.Table.from_arrays([
                pa.array([f'Region_{int(i):04d}' for i in ids], type=pa.string()),
                pa.array(np.arange(comienzo, comienzo + len(inds), dtype=np.int64)),
                pa.array((inds % w).astype(np.int32)),
                pa.array((inds // w).astype(np.int32)),
                lista,
                pa.array([stem] * len(inds), type=pa.string()),
                pa.array(ids, type=pa.int32()),
            ], schema=esquema)
            escritor.write_table(tabla)
        escritor.close()
        escritor = None
    except Exception:
        if escritor is not None:
            escritor.close()
        if parquet.exists():
            parquet.unlink()
        raise

    figura.savefig(jpg, dpi=170, bbox_inches='tight', facecolor='white')
    with csv_r.open('w', newline='', encoding='utf-8-sig') as f:
        writer = csv.DictWriter(f, fieldnames=list(resumen[0]))
        writer.writeheader()
        writer.writerows(resumen)
    with json_p.open('w', encoding='utf-8') as f:
        json.dump({
            'origen_mnf_inverso': str(ruta.resolve()),
            'shape': list(cubo.shape),
            'n_regiones': len(resumen),
            'pixeles_parquet': len(indices),
            'parametros': config,
            'interpretacion': ('Regiones conectadas espacialmente y similares segun SAM; '
                               'no equivalen necesariamente a pigmentos ni a clases verificadas.'),
        }, f, ensure_ascii=False, indent=2)
    return rutas


def ejecutar() -> None:
    p = argparse.ArgumentParser(description=__doc__)
    p.add_argument('--hdr', default=None, help='Un archivo .hdr ENVI de mural (MNF inverso)')
    p.add_argument('--salida', default=None, help='Carpeta DESTINO para resultados')
    p.add_argument('--bandas', type=int, nargs=3, default=[56, 32, 20],
                   metavar=('R', 'G', 'B'), help='Indices de bandas para falso RGB (base 0)')
    p.add_argument('--segmentos', type=int, default=1500,
                   help='Superpixeles SLIC aproximados; NO es numero final de regiones')
    p.add_argument('--compactacion', type=float, default=10.0)
    p.add_argument('--sigma', type=float, default=1.0, help='Suavizado previo a SLIC')
    p.add_argument('--angulo-fusion', type=float, default=4.5,
                   help='SAM maximo en grados para fusionar superpixeles vecinos')
    p.add_argument('--dispersion-max', type=float, default=9.0,
                   help='Cota angular maxima de dispersion acumulada tras fusion')
    p.add_argument('--angulo-pixel', type=float, default=10.0,
                   help='SAM maximo de cada pixel con respecto a su region')
    p.add_argument('--borde', type=int, default=2,
                   help='Pixeles del borde entre regiones que se excluyen')
    p.add_argument('--min-area', type=int, default=250,
                   help='Minimo de pixeles interiores compatibles para guardar region')
    p.add_argument('--max-pixeles-region', type=int, default=4000,
                   help='Tope de espectros por region en Parquet. 0 guarda TODOS.')
    p.add_argument('--semilla', type=int, default=42, help='Semilla del muestreo reproducible')
    p.add_argument('--etiquetas-max', type=int, default=45,
                   help='Cuantas regiones grandes llevaran numero en JPG')
    p.add_argument('--mostrar', action='store_true', help='Mostrar la figura despues de guardarla')
    args = p.parse_args()

    if args.segmentos < 2 or args.min_area < 1 or args.borde < 0 or args.max_pixeles_region < 0:
        p.error('segmentos >=2, min-area >=1, borde >=0 y max-pixeles-region >=0')
    if not (0 < args.angulo_fusion <= 180 and 0 < args.dispersion_max <= 180 and
            0 < args.angulo_pixel <= 180):
        p.error('Los angulos deben estar entre 0 y 180 grados (excluyendo 0).')
    if args.dispersion_max < args.angulo_fusion / 2:
        p.error('dispersion-max es demasiado pequeno frente a angulo-fusion.')

    print('\n' + '=' * 70)
    print('MURAL | SLIC + SAM | REGIONES SIN NUMERO PREFIJADO')
```

```
    print('ARCHIVO EJECUTADO:', Path(__file__).resolve())
    print('NO PIDE PIGMENTOS, CLICS, SEMILLAS, IMAGENES CRUDAS NI SPECTRALON')
    print('=' * 70, flush=True)
    ruta, salida = archivos_entrada_salida(args.hdr, args.salida)
    cubo = cargar_cubo(ruta)
    rgb = falso_rgb(cubo, tuple(args.bandas))
    mapa, nslic = mapa_inicial(cubo, rgb, args.segmentos, args.compactacion,
                               args.sigma, args.angulo_fusion, args.dispersion_max)
    mapa, resumen = mascaras_espectrales(cubo, mapa, args.borde,
                                         args.angulo_pixel, args.min_area)
    if not resumen:
        print('No se detectaron regiones que cumplan los umbrales. No se guardo Parquet.')
        print('Prueba reducir --min-area, --borde o aumentar --angulo-pixel.')
        return
    indices = elegir_pixeles(mapa, resumen, args.max_pixeles_region, args.semilla)
    fig = figura_resultado(rgb, mapa, resumen, args.etiquetas_max)
    parametros = vars(args).copy()
    parametros['superpixeles_realmente_generados'] = nslic
    try:
        rutas = guardar(ruta, salida, cubo, mapa, resumen, indices, fig, parametros)
        print(f'Listo: {len(resumen)} regiones, {len(indices):,} espectros guardados.', flush=True)
        for archivo in rutas:
            print(' -', archivo, flush=True)
        if args.mostrar:
            import matplotlib.pyplot as plt
            plt.show(block=False)
            plt.pause(.2)
    finally:
        import matplotlib.pyplot as plt
        if not args.mostrar:
            plt.close(fig)


if __name__ == '__main__':
    ejecutar()
```

Parte II. Explicación completa para estudiar y presentar

La explicación siguiente reproduce el desarrollo paso a paso de nuestra conversación e incorpora la imagen de resultado del mural para facilitar el repaso.

Explicación del algoritmo de segmentación automática SLIC + SAM para imágenes hiperespectrales de murales

Vamos a estudiar el código `Segmentacion_Mural_SLIC_SAM.py` que utilizaste para obtener las **35 regiones de tu mural**.

Te explicaré qué hace cada etapa, cuál es su fundamento matemático, por qué utilizamos esos métodos y cómo puedes presentarlo ante tu asesor.

Voy a distinguir entre lo que efectivamente implementa el programa y la interpretación física de sus resultados.

---

1. ¿Qué problema estamos intentando resolver?

Partamos de tu objetivo.

Tienes una imagen hiperespectral de un mural que ya pasó por el siguiente procesamiento:

$$\boxed{\text{Normalización} \longrightarrow \text{MNF} \longrightarrow \text{MNF inverso}}$$

Ahora quieres identificar automáticamente regiones que tengan características espectrales similares para posteriormente analizar sus espectros con una CNN.

Antes realizabas la selección de regiones manualmente. El inconveniente es que ese procedimiento requiere intervención humana y puede ser complicado cuando se trabaja con imágenes grandes.

El objetivo de este programa es automatizar la selección de regiones espectralmente homogéneas.

Para conseguirlo, combinamos dos técnicas:

- **SLIC (Simple Linear Iterative Clustering):** divide la imagen en pequeñas regiones espacialmente coherentes.
- **SAM (Spectral Angle Mapper):** mide qué tan similares son dos espectros hiperespectrales.

La idea fundamental es:

> Primero dividimos la imagen en pequeñas regiones utilizando SLIC. Después analizamos sus espectros con SAM y fusionamos las regiones vecinas que sean suficientemente similares.

Finalmente, eliminamos píxeles que podrían contaminar los espectros representativos.

2. Primera etapa: cargar el cubo hiperespectral

¿Qué recibe el programa?

El código recibe una imagen hiperespectral en formato ENVI, mediante su archivo `.hdr`.

En tu caso, esa imagen ya fue normalizada, transformada mediante MNF y reconstruida con MNF inverso.

El programa **no realiza nuevamente ninguna de esas operaciones**.

Matemáticamente, podemos representar tu imagen como:

$$\boxed{I(x, y, \lambda)}$$

Donde:

- x: posición horizontal.
- y: posición vertical.
- λ: longitud de onda.

Supongamos que tu imagen tiene dimensiones:

$$1392 \times 1392 \times 128$$

Esto significa que tenemos aproximadamente 1.94 millones de píxeles y que cada uno contiene 128 valores espectrales.

Podemos representar el espectro de un píxel como:

$$\mathbf{s}_{x,y} = \begin{bmatrix} R(\lambda_1) \\ R(\lambda_2) \\ \vdots \\ R(\lambda_{128}) \end{bmatrix}$$

Cada componente corresponde a la reflectancia en una banda espectral, suponiendo que tu cubo reconstruido conserva esa magnitud.

¿Por qué es importante?

Porque no estamos trabajando únicamente con el color visible de cada píxel.

Tenemos información espectral que nos permite comparar materiales incluso cuando visualmente parecen similares.

3. Segunda etapa: construir una imagen de falso RGB

El algoritmo SLIC necesita una representación visual de la imagen.

Para ello, el programa selecciona tres bandas del cubo hiperespectral.

En el código aparecen estos valores:

```
p.add_argument(
    '--bandas',
    type=int,
    nargs=3,
    default=[56, 32, 20]
)
```

Esto significa que seleccionamos las bandas con índices:

$$R = 56, \qquad G = 32, \qquad B = 20$$

Importante: estos números son índices de bandas, no longitudes de onda en nanómetros.

Además, el programa normaliza cada canal utilizando sus percentiles 2 y 98.

La operación es aproximadamente:

$$I_{\text{normalizada}} = \text{clip}\left(\frac{I - P_2}{P_{98} - P_2}, 0, 1\right)$$

Donde:

- P_2: percentil 2 de los valores de esa banda.
- P_{98}: percentil 98.
- clip: limita el resultado al intervalo $[0, 1]$.

Esto mejora el contraste de la imagen utilizada para segmentar.

¿Por qué no utilizamos las 128 bandas directamente?

Porque en esta implementación SLIC trabaja con una imagen de tres canales.

Esto reduce la complejidad del proceso inicial.

Sin embargo, hay algo fundamental:

Aunque SLIC utiliza tres bandas, SAM utiliza las 128 bandas espectrales.

Por tanto, la comparación espectral posterior aprovecha la información hiperespectral.

Cómo explicárselo a tu asesor

> «Primero genero una representación de falso RGB para realizar una segmentación espacial inicial. Esta representación únicamente se utiliza para SLIC; las comparaciones espectrales posteriores se realizan sobre el cubo hiperespectral completo».

4. Tercera etapa: segmentación mediante SLIC

Esta es la primera parte importante del algoritmo.

4.1. ¿Qué es SLIC?

SLIC significa *Simple Linear Iterative Clustering*.

Es un algoritmo que divide una imagen en pequeñas regiones llamadas **superpíxeles**.

Un superpíxel es un conjunto de píxeles vecinos que presentan características visuales similares.

Imagina que tenemos una esfera azul.

En lugar de analizar inmediatamente todos sus píxeles individualmente, SLIC divide su superficie en pequeñas regiones.

La ventaja es que podemos comenzar trabajando con cientos o miles de regiones, en lugar de hacerlo directamente con millones de píxeles.

4.2. ¿Cómo funciona matemáticamente?

SLIC utiliza una variante de K-Means que considera simultáneamente información de color y posición espacial.

Para ello, representa cada píxel mediante cinco características:

$$\boxed{\mathbf{p}_i = (L_i, a_i, b_i, x_i, y_i)}$$

Las primeras tres corresponden al espacio de color CIELAB:

- L: luminosidad.
- a: componente cromática verde-rojo.
- b: componente cromática azul-amarillo.

Las últimas dos corresponden a la posición espacial.

SLIC busca agrupar píxeles que estén próximos tanto en color como en posición.

Distancia de color

La distancia entre dos píxeles puede expresarse como:

$$d_c = \sqrt{(L_i - L_j)^2 + (a_i - a_j)^2 + (b_i - b_j)^2}$$

Distancia espacial

También calcula:

$$d_s = \sqrt{(x_i - x_j)^2 + (y_i - y_j)^2}$$

Distancia combinada

Una forma habitual de expresar la distancia utilizada por SLIC es:

$$\boxed{D = \sqrt{d_c^2 + \left(\frac{m}{S}\right)^2 d_s^2}}$$

Donde:

- m: parámetro de compactación.
- S: separación espacial aproximada entre centros de superpíxeles.
- d_c: distancia de color.
- d_s: distancia espacial.

Esta formulación describe el principio de SLIC; la implementación concreta de `scikit-image` incorpora detalles adicionales de normalización y optimización.

4.3. ¿Qué significa el parámetro de compactación?

En nuestro código:

```
p.add_argument('--compactacion', type=float, default=10.0)
```

La compactación controla la importancia relativa de la cercanía espacial.

Compactación elevada: favorece superpíxeles más regulares.

Compactación baja: permite que las regiones se adapten más a las variaciones de color.

En un mural, esto importa porque los límites entre materiales suelen ser irregulares.

Por eso, queremos que los superpíxeles puedan adaptarse razonablemente a esos límites.

4.4. ¿Cuántos superpíxeles genera?

En el código:

```
p.add_argument('--segmentos', type=int, default=1500)
```

Esto significa que solicitamos aproximadamente 1500 superpíxeles iniciales.

No significa que el programa vaya a encontrar 1500 pigmentos.

Tampoco garantiza exactamente 1500 superpíxeles: el número real depende de la imagen y de la implementación de SLIC.

Este es el primer punto importante para tu presentación:

SLIC genera una sobresegmentación inicial.

Todavía no hemos identificado las regiones definitivas.

5. Cuarta etapa: calcular un espectro representativo para cada superpíxel

Ahora tenemos aproximadamente 1500 regiones pequeñas.

El siguiente paso consiste en caracterizarlas espectralmente.

Aquí es donde volvemos a utilizar las 128 bandas del cubo hiperespectral.

5.1. ¿Cómo obtenemos el espectro de una región?

Supongamos que un superpíxel contiene N píxeles.

Cada uno tiene un espectro:

$$\mathbf{s}_1, \mathbf{s}_2, \dots, \mathbf{s}_N$$

Calculamos el espectro promedio:

$$\boxed{\bar{\mathbf{s}} = \frac{1}{N} \sum_{i=1}^{N} \mathbf{s}_i}$$

Esta operación se realiza componente por componente.

Por ejemplo, para la banda 1:

$$\overline{R}(\lambda_1) = \frac{1}{N} \sum_{i=1}^{N} R_i(\lambda_1)$$

Y repetimos el procedimiento para las 128 bandas.

Así obtenemos un vector:

$$\bar{\mathbf{s}} \in \mathbb{R}^{128}$$

que representa el comportamiento espectral promedio de ese superpíxel.

Un detalle importante del código

El programa no utiliza necesariamente todos los píxeles para calcular estos promedios iniciales.

Para controlar el consumo de memoria, toma una muestra espacial regular de la imagen, con un máximo aproximado de 300 000 posiciones muestreadas.

Por tanto, los espectros representativos iniciales son **estimaciones basadas en un muestreo**.

Esto es relevante si tu asesor pregunta por la eficiencia computacional.

6. Quinta etapa: normalización de los vectores espectrales

Después de calcular los espectros promedio, el programa normaliza cada vector.

Matemáticamente:

$$\boxed{\hat{\mathbf{s}} = \frac{\bar{\mathbf{s}}}{\|\bar{\mathbf{s}}\|}}$$

Donde:

$$\|\bar{\mathbf{s}}\| = \sqrt{\sum_{j=1}^{128} \bar{s}_j^2}$$

Esto hace que:

$$\|\hat{\mathbf{s}}\| = 1$$

¿Por qué hacemos esto?

Porque queremos comparar principalmente la **forma de los espectros**, no solamente su magnitud.

Imagina dos espectros:

$$\mathbf{s}_1 = (1, 2, 3)$$

$$\mathbf{s}_2 = (2, 4, 6)$$

El segundo tiene el doble de intensidad que el primero.

Sin embargo, ambos tienen exactamente la misma forma.

Al normalizarlos:

$$\hat{\mathbf{s}}_1 = \hat{\mathbf{s}}_2$$

Por tanto, podemos reconocer que presentan el mismo comportamiento relativo.

Esto es particularmente útil cuando existen diferencias de intensidad relacionadas con iluminación o geometría.

Pero hay una limitación: dos materiales diferentes pueden presentar espectros con formas similares. Por tanto, una similitud angular elevada no demuestra que sean químicamente idénticos.

7. Sexta etapa: comparación espectral mediante SAM

Esta es la segunda parte central del programa.

7.1. ¿Qué es SAM?

SAM significa *Spectral Angle Mapper*.

Es una técnica que mide la similitud entre dos espectros calculando el ángulo que forman sus vectores en un espacio multidimensional.

En nuestro caso, cada espectro es un vector de 128 dimensiones.

La expresión matemática es:

$$\boxed{\theta = \cos^{-1}\left(\frac{\mathbf{s}_1 \cdot \mathbf{s}_2}{\|\mathbf{s}_1\|\|\mathbf{s}_2\|}\right)}$$

Donde:

- $\mathbf{s}_1$ y $\mathbf{s}_2$ son los espectros.
- $\mathbf{s}_1 \cdot \mathbf{s}_2$ es su producto escalar.
- $\|\mathbf{s}_1\|$ y $\|\mathbf{s}_2\|$ son sus normas.
- θ es el ángulo espectral.

Como nuestros vectores ya están normalizados, la expresión se simplifica:

$$\boxed{\theta = \cos^{-1}\left(\hat{\mathbf{s}}_1 \cdot \hat{\mathbf{s}}_2\right)}$$

7.2. Interpretación física

Imagina que tenemos dos espectros correspondientes a dos regiones del mural.

Si ambos tienen una forma muy similar, el ángulo entre sus vectores será pequeño.

Si tienen comportamientos espectrales diferentes, el ángulo será mayor.

Por ejemplo:

Ángulo SAM	Interpretación orientativa
0°	Espectros proporcionales con el mismo sentido
2°	Formas espectrales muy similares

Ángulo SAM	Interpretación orientativa
5°	Diferencia angular moderada
15°	Diferencia angular mayor
30°	Diferencia angular considerable

Estos valores no constituyen una escala universal de identificación de materiales. Su significado depende del instrumento, las bandas y la variabilidad de los datos.

Ejemplo matemático sencillo

Supongamos dos espectros de tres bandas:

$$\mathbf{s}_1 = (1, 2, 3)$$

$$\mathbf{s}_2 = (2, 4, 6)$$

Su producto escalar es:

$$\mathbf{s}_1 \cdot \mathbf{s}_2 = 2 + 8 + 18 = 28$$

Sus normas son:

$$\|\mathbf{s}_1\| = \sqrt{14}$$

$$\|\mathbf{s}_2\| = \sqrt{56}$$

Entonces:

$$\cos\theta = \frac{28}{\sqrt{14}\sqrt{56}} = 1$$

Por tanto:

$$\boxed{\theta = 0^\circ}$$

Aunque las intensidades son diferentes, los espectros son proporcionales.

Eso es precisamente lo que SAM puede reconocer.

8. Séptima etapa: encontrar qué regiones son vecinas

Aquí hay un detalle muy importante del programa.

No comparamos indiscriminadamente todos los superpíxeles para fusionarlos.

Primero identificamos cuáles comparten una frontera.

El código utiliza la función:

```
pares_adyacentes(etiquetas)
```

Esta función examina las etiquetas de píxeles contiguos horizontal y verticalmente.

Si dos superpíxeles comparten una frontera, los considera vecinos.

Matemáticamente, podemos representar esta información mediante un grafo:

$$G = (V, E)$$

Donde:

- V: conjunto de superpíxeles.
- E: conexiones entre superpíxeles vecinos.

Cada superpíxel es un nodo.

Dos nodos están conectados si sus regiones comparten una frontera.

¿Por qué hacemos esto?

Imagina dos esferas azules ubicadas en lugares diferentes del mural.

Aunque tengan espectros similares, no queremos necesariamente convertirlas en una sola región espacial.

Por eso, la fusión se restringe a regiones conectadas.

Esto permite que el algoritmo produzca regiones espacialmente coherentes.

9. Octava etapa: fusión de regiones mediante SAM

Llegamos al corazón del algoritmo.

Ya tenemos:

1. Superpíxeles iniciales.
2. Un espectro representativo para cada uno.
3. Información sobre cuáles son vecinos.

Ahora decidimos cuáles deben fusionarse.

9.1. Primer criterio: similitud espectral

En el código encontramos:

```
p.add_argument(
    '--angulo-fusion',
    type=float,
    default=4.5
)
```

Esto establece un umbral:

$$\boxed{\theta_{\text{fusión}} = 4.5^\circ}$$

Dos regiones vecinas solamente pueden fusionarse si:

$$\theta(\mathbf{s}_i, \mathbf{s}_j) \leq 4.5^\circ$$

Por ejemplo:

Caso A

$$\theta = 2.1^\circ$$

La pareja supera el primer criterio y puede considerarse para fusión.

Caso B

$$\theta = 8.7^\circ$$

La pareja no se fusiona.

Pero el código utiliza un segundo criterio.

9.2. Segundo criterio: dispersión espectral acumulada

El código contiene:

```
p.add_argument(
    '--dispersion-max',
    type=float,
    default=9.0
)
```

¿Por qué necesitamos este parámetro?

Imagina tres regiones:

$$A,\ B,\ C$$

Supongamos que:

$$\theta(A, B) = 3^\circ$$

$$\theta(B, C) = 3^\circ$$

Pero:

$$\theta(A, C) = 6^\circ$$

Si únicamente consideráramos la similitud entre vecinos, podríamos terminar fusionando regiones que, en conjunto, presentan diferencias espectrales considerables.

Este fenómeno puede provocar uniones sucesivas entre materiales diferentes.

Para limitarlo, el algoritmo lleva un registro de la dispersión angular acumulada de cada región fusionada.

¿Cómo se calcula en nuestro código?

Cuando se fusionan dos regiones, se calcula un nuevo vector representativo a partir de sus vectores acumulados.

Después se mide cuánto se desplaza angularmente el representante de cada región hacia el nuevo representante.

El código actualiza una cota:

$$\boxed{r_{\text{nuevo}} = \max\left(r_A + \Delta\theta_A,\, r_B + \Delta\theta_B\right)}$$

Donde:

- r_A y r_B son las cotas angulares acumuladas.
- $\Delta\theta_A$ es el desplazamiento del representante de A.
- $\Delta\theta_B$ es el desplazamiento del representante de B.

La fusión se acepta únicamente cuando:

$$\boxed{r_{\text{nuevo}} \leq 9^\circ}$$

Esta es una **cota conservadora calculada durante las fusiones**, no una medición exacta de la dispersión de todos los píxeles de la región.

¿Qué conseguimos?

Reducimos el riesgo de que varias fusiones sucesivas produzcan regiones excesivamente heterogéneas.

10. Novena etapa: ¿cómo decide el orden de las fusiones?

Este detalle también es interesante desde el punto de vista computacional.

El programa utiliza una estructura llamada *cola de prioridad*.

En Python se implementa mediante:

```
import heapq
```

El algoritmo calcula inicialmente los ángulos SAM entre superpíxeles vecinos y los ordena de manera que pueda procesar primero las parejas más similares.

El procedimiento es:

1. Identifica la pareja de regiones vecinas con menor ángulo SAM.
2. Comprueba los criterios de fusión.
3. Si se cumplen, fusiona las regiones.
4. Recalcula el espectro representativo de la región resultante.
5. Actualiza sus conexiones con las regiones vecinas.
6. Repite el proceso.

Este procedimiento se conoce como **fusión aglomerativa de regiones**.

A diferencia de K-Means, no necesita establecer previamente el número final de grupos.

¿Cuándo termina?

Cuando ya no quedan fusiones admisibles según los criterios establecidos.

Por eso, el algoritmo puede producir 20, 35, 50 o cualquier otro número de regiones.

En tu mural produjo 35 regiones finales después de aplicar los filtros posteriores.

Ese número no fue introducido manualmente.

Surgió de la estructura espacial y espectral de la imagen, junto con los parámetros que elegimos.

11. Décima etapa: eliminar píxeles de los bordes

Una vez que tenemos las regiones fusionadas, el programa comienza a seleccionar los píxeles que conservará.

El primer filtro consiste en eliminar píxeles próximos a las fronteras entre regiones.

En el código:

```
p.add_argument('--borde', type=int, default=2)
```

Esto significa que excluimos una franja alrededor de las fronteras.

¿Por qué?

Porque los píxeles cercanos a los límites entre materiales pueden contener contribuciones de ambos.

Por ejemplo, en el límite entre una esfera azul y una esfera amarilla, algunos píxeles podrían contener una combinación de ambas señales.

Podemos expresar conceptualmente un espectro mezclado como:

$$\mathbf{s}_{\text{mezcla}} = \alpha \mathbf{s}_{\text{azul}} + (1-\alpha)\mathbf{s}_{\text{amarillo}}$$

Donde:

$$0 \leq \alpha \leq 1$$

Esta ecuación representa un modelo sencillo de mezcla lineal. No implica que todas las mezclas reales de pigmentos sigan ese comportamiento.

¿Cómo elimina los bordes?

El código utiliza filtros de mínimo y máximo sobre las etiquetas de las regiones.

Para un píxel, examina una vecindad cuadrada.

Si todas las etiquetas dentro de esa vecindad son iguales, el píxel se considera interior.

En caso contrario, se descarta.

Con:

$$b = 2$$

la vecindad tiene dimensiones:

$$(2b+1) \times (2b+1) = 5 \times 5$$

Por tanto, un píxel debe encontrarse dentro de una zona de etiquetas uniformes de 5×5 para superar este filtro.

Esto ayuda a reducir la contaminación de los espectros por fronteras entre regiones.

12. Undécima etapa: filtrado espectral de cada píxel

Hasta ahora, el algoritmo fusionó regiones utilizando sus espectros representativos.

Pero dentro de una región todavía puede haber píxeles con espectros diferentes.

Por eso, realizamos una segunda evaluación SAM.

Esta vez no comparamos regiones entre sí.

Comparamos cada píxel con el espectro representativo de su propia región.

12.1. Calcular la referencia espectral

Para cada región calculamos nuevamente un espectro promedio, utilizando una muestra de sus píxeles interiores válidos.

Lo normalizamos:

$$\hat{\mathbf{s}}_R = \frac{\bar{\mathbf{s}}_R}{\|\bar{\mathbf{s}}_R\|}$$

12.2. Comparar cada píxel

Para cada píxel p:

$$\boxed{\theta_p = \cos^{-1}\left(\frac{\mathbf{s}_p \cdot \bar{\mathbf{s}}_R}{\|\mathbf{s}_p\|\|\bar{\mathbf{s}}_R\|}\right)}$$

El código establece:

```
p.add_argument(
    '--angulo-pixel',
    type=float,
    default=10.0
)
```

Por tanto, conservamos el píxel si:

$$\boxed{\theta_p \leq 10^\circ}$$

Si supera ese ángulo, se descarta.

Ejemplo

Supongamos que una región contiene 10 000 píxeles.

Después del filtrado:

- 800 píxeles se eliminan por encontrarse cerca de los bordes.
- 500 píxeles adicionales se eliminan por no superar el criterio SAM.
- 8700 píxeles permanecen.

Estos números son ilustrativos.

El resultado es un conjunto de espectros más consistente con el espectro representativo de la región.

Una limitación

Este filtrado no garantiza que los píxeles restantes pertenezcan al mismo pigmento.

Solamente garantiza que cumplen un criterio de similitud angular respecto a la referencia calculada.

13. Duodécima etapa: eliminar regiones pequeñas

El programa utiliza otro parámetro:

```
p.add_argument('--min-area', type=int, default=250)
```

Esto significa que una región necesita conservar al menos:

$$\boxed{N_{\text{píxeles}} \geq 250}$$

después de los filtros para mantenerse en el resultado.

¿Por qué?

Porque algunas regiones pequeñas podrían corresponder a ruido, detalles finos, variaciones locales o zonas de transición.

Al excluirlas, buscamos trabajar con regiones que tengan una cantidad suficiente de espectros.

Sin embargo, hay una consecuencia:

También podemos eliminar regiones pequeñas que correspondan a materiales reales.

Por eso, el tamaño mínimo es un parámetro metodológico que tendremos que justificar.

14. Decimotercera etapa: asignar etiquetas automáticamente

Una vez finalizados los filtros, el programa enumera las regiones conservadas.

Por ejemplo:

```
Region_0001
Region_0002
Region_0003
...
Region_0035
```

El código ordena las regiones por cantidad de píxeles conservados, de mayor a menor.

Esto significa que:

- `Region_0001` corresponde a la región con más píxeles conservados.
- `Region_0002` corresponde a la segunda.
- Y así sucesivamente.

Los números no indican pigmentos ni tienen significado químico.

Tampoco necesariamente se conservan entre ejecuciones si modificamos los parámetros.

¿Qué representa entonces una región?

Una región es un conjunto espacialmente conectado durante la etapa de fusión, cuyos píxeles conservados cumplen los criterios geométricos y espectrales.

Hay un matiz: después de eliminar bordes y píxeles atípicos, la máscara final de una región podría quedar dividida en componentes desconectados. El código no vuelve a comprobar la conectividad en esa última etapa.

15. Decimocuarta etapa: seleccionar espectros para exportar

Una vez identificadas las regiones, el programa prepara los datos que posteriormente podremos analizar.

En el código encontramos:

```
p.add_argument(
    '--max-pixeles-region',
    type=int,
    default=4000
)
```

Esto significa que el programa guarda un máximo de 4000 espectros por región.

Si una región contiene menos de 4000 píxeles válidos, guarda todos.

Si contiene más, selecciona aleatoriamente 4000, sin reemplazo.

Utiliza una semilla fija:

```
p.add_argument('--semilla', type=int, default=42)
```

Esto permite reproducir el muestreo cuando se utilizan los mismos datos y parámetros.

¿Por qué limitar el número de espectros?

Principalmente para:

- Reducir el tamaño del archivo Parquet.
- Controlar el consumo de memoria.
- Evitar que las regiones grandes dominen excesivamente el conjunto exportado.

Por ejemplo, si tenemos 35 regiones y todas conservan al menos 4000 píxeles, el máximo exportado será:

$$35 \times 4000 = 140\,000$$

espectros.

Cada espectro contiene 128 bandas.

Esto representa:

$$140\,000 \times 128 = 17\,920\,000$$

valores espectrales.

Importante: el límite de 4000 espectros afecta al Parquet, no a la máscara de regiones que aparece en el JPG.

16. Decimoquinta etapa: guardar los resultados

Finalmente, el programa genera cuatro archivos.

A. Archivo Parquet

Es el archivo principal para trabajar posteriormente con los espectros.

Contiene columnas como:

Columna	Significado
`clase`	Etiqueta provisional de región
`pixel_id`	Identificador del espectro exportado
`coord_x`	Coordenada horizontal
`coord_y`	Coordenada vertical
`espectro`	Vector de 128 valores
`imagen_id`	Identificador de la imagen original
`region_id`	Identificador numérico de región

Este archivo puede utilizarse como entrada para una etapa posterior de clasificación.

Pero las etiquetas `Region_0001`, etc., no deben utilizarse como si fueran clases químicas verificadas para entrenar una CNN supervisada.

B. Archivo JPG

Es la imagen que me compartiste.

Muestra:

- A la izquierda, el mural en falso RGB.
- A la derecha, las regiones detectadas.
- Colores artificiales para distinguir regiones.
- Identificadores numéricos.

Su principal utilidad es evaluar visualmente la segmentación.

C. Archivo CSV

Contiene un resumen de cada región.

En esta versión del código incluye:

- Identificador de región.
- Etiqueta provisional.
- Píxeles de la región antes del filtrado final.
- Píxeles compatibles conservados.
- Píxeles exportados.
- Coordenadas del centro.
- Límites espaciales de la región.

El CSV no incluye directamente una medida de dispersión SAM por región. Esa sería una mejora posible para una versión posterior.

D. Archivo JSON

Guarda información necesaria para reproducir el experimento:

- Ruta de la imagen.
- Dimensiones del cubo.
- Número de regiones.
- Número de espectros exportados.
- Parámetros de segmentación.

17. ¿Cómo interpretar el resultado de tu mural?

Volvamos a la imagen que obtuviste.

Tu programa detectó **35 regiones**.

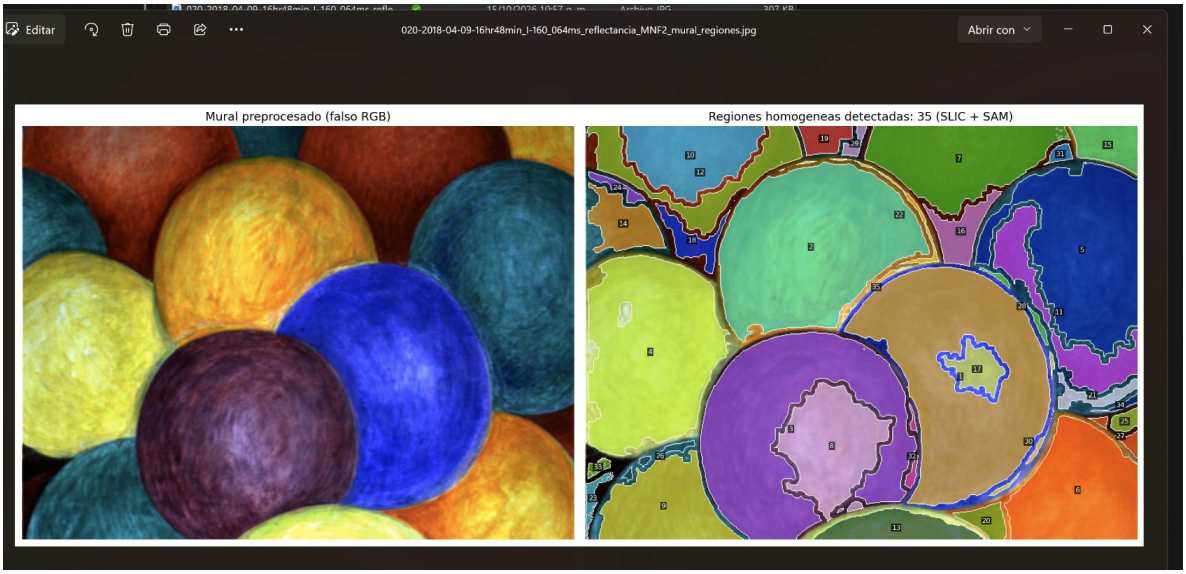


Figura 1: Resultado real del mural: falso RGB (izquierda) y regiones automáticas SLIC + SAM (derecha).

En general, observamos que varias regiones coinciden razonablemente con las superficies de las esferas.

Por ejemplo, la región azul intensa de la derecha aparece separada de la región verde situada encima.

También observamos que algunas esferas quedan divididas en regiones interiores y exteriores.

Esto podría ocurrir por distintas razones:

1. Variaciones de reflectancia dentro de un mismo material.
2. Diferencias de iluminación.
3. Textura de la superficie.
4. Presencia de distintos materiales.
5. Efectos de los umbrales de segmentación.

Con la imagen JPG no podemos determinar cuál de esas explicaciones es la correcta.

Necesitaríamos analizar los espectros de las regiones.

Algo que conviene mencionar ante tu asesor

El algoritmo no está diseñado para identificar objetos completos.

Por ejemplo, no sabe qué es una esfera.

Su objetivo es encontrar regiones que cumplan criterios de similitud espectral y proximidad espacial.

Por eso, una esfera puede dividirse en varias regiones si presenta variaciones espectrales suficientemente importantes.

Esto no necesariamente representa un fallo de segmentación.

Depende de si nuestro objetivo es identificar objetos visuales o materiales espectralmente homogéneos.

Para tu investigación, nos interesa principalmente lo segundo.

18. ¿Cómo se relaciona con tus dos flujos de trabajo?

Esta es probablemente la parte más importante para tu asesor.

Actualmente estás comparando:

Flujo 1

$$\boxed{\text{Normalización} \rightarrow \text{MNF} \rightarrow \text{MNF}^{-1} \rightarrow \text{CNN}}$$

Flujo 2

$$\boxed{\text{Normalización} \rightarrow \text{MNF} \rightarrow \text{MNF}^{-1} \rightarrow \text{Autoencoder} \rightarrow \text{CNN}}$$

La segmentación automática puede funcionar como una etapa común de selección de píxeles.

Por ejemplo:

$$\text{MNF}^{-1} \rightarrow \boxed{\text{SLIC + SAM}} \rightarrow \text{Espectros seleccionados}$$

Después, esos espectros pueden seguir cualquiera de las dos rutas.

En el primer flujo, la CNN recibe los espectros reconstruidos.

En el segundo, el autoencoder transforma los espectros a su representación latente, que posteriormente recibe la CNN.

¿Por qué es metodológicamente interesante?

Porque podemos utilizar exactamente los mismos píxeles seleccionados para evaluar ambos modelos.

Así reducimos una posible fuente de variación experimental.

Si cada flujo utilizara regiones distintas, sería más difícil determinar si una diferencia en el rendimiento se debe al autoencoder o a la selección de datos.

Mi recomendación es que ambos flujos utilicen el mismo conjunto de píxeles y las mismas particiones de evaluación.

Además, debemos evitar que píxeles muy próximos de una misma región aparezcan simul-

táneamente en entrenamiento y prueba si eso introduce dependencia espacial que infle las métricas.

19. ¿Qué limitaciones tiene el algoritmo actual?

Te recomiendo comentar estas limitaciones con tu asesor. Son importantes científicamente.

Limitación	Consecuencia
SLIC utiliza solamente tres bandas	Puede ignorar diferencias visibles en otras bandas
SAM compara principalmente formas espectrales	Puede considerar similares materiales distintos
Los umbrales son experimentales	El número de regiones depende de los parámetros
Se utilizan promedios espectrales muestreados	Las referencias pueden no representar perfectamente toda la región
Se excluyen regiones pequeñas	Podríamos perder materiales presentes en áreas reducidas
No existe identificación química	Las regiones no equivalen directamente a pigmentos
No hay validación cuantitativa de segmentación	Todavía no podemos afirmar qué tan preciso es el método

Una cuestión especialmente importante es que los parámetros actuales —4.5°, 9° y 10°— son valores iniciales.

No hemos demostrado todavía que sean óptimos para tu sistema de adquisición o tus materiales.

Por tanto, no conviene presentarlos como valores físicamente establecidos.

20. ¿Qué podrías presentar como avances?

Yo organizaría tu exposición en cinco partes.

1. Problema

La selección manual de regiones en imágenes hiperespectrales requiere tiempo e intervención humana.

Además, se vuelve difícil de aplicar sistemáticamente a murales con geometrías complejas.

2. Propuesta

Desarrollar un procedimiento de segmentación automática que combine información espacial y espectral.

La metodología utiliza SLIC para generar superpíxeles y SAM para fusionar regiones espectralmente similares.

3. Implementación

Se desarrolló un programa en Python que recibe imágenes hiperespectrales reconstruidas mediante MNF inverso.

El procedimiento incluye:

- Segmentación inicial.

- Extracción de espectros representativos.
- Fusión espectral de regiones vecinas.
- Exclusión de bordes.
- Filtrado espectral.
- Exportación de datos.

4. Resultados preliminares

Se aplicó el procedimiento a una región de mural.

El algoritmo identificó automáticamente **35 regiones**, sin establecer previamente ese número.

La inspección visual muestra una correspondencia razonable entre varias regiones detectadas y las superficies del mural.

Sin embargo, algunas superficies aparecen subdivididas.

5. Trabajo pendiente

Evaluar cuantitativamente la homogeneidad espectral de las regiones y estudiar la sensibilidad de los resultados a los parámetros del algoritmo.

Posteriormente, utilizar los espectros seleccionados como entradas para los modelos de clasificación.

21. Una explicación breve que puedes darle a tu asesor

Te propongo explicar el procedimiento de esta manera:

> Para automatizar la selección de regiones en imágenes hiperespectrales, desarrollé un algoritmo que combina SLIC y SAM.
>
> Primero, SLIC divide la imagen en superpíxeles utilizando información de color y proximidad espacial.
>
> Posteriormente, calculo un espectro promedio de cada superpíxel utilizando las 128 bandas del cubo hiperespectral.
>
> Con SAM comparo los espectros de regiones vecinas. Cuando el ángulo espectral es suficientemente pequeño y la fusión no supera un límite de dispersión acumulada, las regiones se unen.
>
> Este procedimiento continúa hasta que ya no existen fusiones admisibles, por lo que no necesito especificar previamente el número de regiones.
>
> Después elimino píxeles cercanos a las fronteras, descarto píxeles espectralmente incompatibles y excluyo regiones demasiado pequeñas.
>
> Finalmente, guardo los espectros seleccionados en un archivo Parquet, conservando sus coordenadas y las etiquetas de las regiones.
>
> En una primera aplicación sobre un mural, el algoritmo detectó 35 regiones automáticamente.
>
> El siguiente paso será evaluar su homogeneidad espectral y estudiar cómo integrar estos resultados en los dos flujos de clasificación que estoy comparando.

22. Tres preguntas que probablemente te haga tu asesor

Pregunta 1. ¿Por qué utilizaste SLIC y SAM juntos?

Porque resuelven problemas complementarios.

SLIC incorpora información espacial y visual para generar regiones iniciales.

SAM incorpora la información espectral de las 128 bandas para decidir cuáles de esas regiones pueden fusionarse.

La combinación permite trabajar con regiones espacialmente coherentes y espectralmente similares.

Pregunta 2. ¿Cómo sabes que realmente identificaste pigmentos?

Todavía no podemos afirmarlo.

El algoritmo identifica regiones que cumplen criterios de homogeneidad espectral, pero no determina su composición química.

Para identificar pigmentos necesitamos comparar sus espectros con referencias verificadas o aplicar un modelo de clasificación entrenado con esas referencias.

Pregunta 3. ¿Cómo sabes que 35 regiones es el número correcto?

No lo sabemos todavía.

El número 35 es el resultado de aplicar los parámetros actuales a esa imagen.

Para evaluar su pertinencia necesitamos estudiar la homogeneidad espectral de las regiones, revisar sus límites y comparar los resultados obtenidos con diferentes umbrales.

23. Lo que considero más importante para tu tesis

Hay una diferencia entre decir:

«Desarrollé un algoritmo que identifica automáticamente pigmentos».

Y decir:

«Desarrollé un algoritmo que segmenta automáticamente imágenes hiperespectrales en regiones espacialmente coherentes y espectralmente similares, con el objetivo de facilitar la extracción de espectros para su posterior clasificación».

La segunda afirmación describe correctamente lo que hace tu programa y es científicamente más defendible.

Además, abre una posibilidad interesante para tu investigación: **evaluar si la selección automática de regiones homogéneas mejora la confiabilidad de los espectros que reciben tus modelos de clasificación.**

Recomendación para la reunión

Presenta la imagen con las 35 regiones, explica el fundamento de SLIC y SAM, y deja claro que los resultados son preliminares.

El siguiente avance importante no debería ser simplemente obtener una segmentación visualmente más bonita, sino demostrar cuantitativamente que las regiones seleccionadas presentan una homogeneidad espectral adecuada.

Eso convertiría tu procedimiento de selección automática en una metodología que puedes justificar y evaluar dentro de tu tesis.